



Python codes for exam

Introduction to python (Universiteit van Amsterdam)



Scan to open on Studocu

Dictionary: d.update({key:value}) new_key=sample.keys()-['age'] → removes the key age from dict *arg - undefined nr of arguments **kwargs - undefined nr of key value pars Avengers=['a', 'b'] E=enumerate(avengers) Print(list(e)) → [(0, 'a'), (1, 'b')] Sorted - first capital letters then lowercase Strings: String[3] → trecia string'o indeksa paduoda String.lower(), string.upper(), string.capitalize() Splitting: S1='axbcxg h\nij' S1.split('x',10) → ['a', 'bc','g h\nij"] Joining: L1=['a', 'b'] 'xy'.join(l1) → 'axyb' "".join(l1) → 'ab' Striping: Strip everything from the sides until finds a character that is not in brackets S1.strip('_f') → 'gedee' S1.strip('f') → '___fgedeef___' Replace: 'abab' S1.replace('ab', 'xx', 10) → 'xxxx' S1.replace('m', 'xx') → 'abab' Find: S1='abcdabcd' S1.find('b') → 1 S1.find('b', 3) → 5 S1.find('b', 2, 5) → -1 S1.index('b') → 1 S1.index('e') → error Count: s1 = 'abcdabcd' S1.count('b') → 2 S1.count('b', 3, 7) → 1 Iterpimas i strings: Print(f"okay is slay") → okay yra variable Print(f"okay:r is slay") → okay yra variable !r string with quotes, !s string version, !a same as !r but escape non-ASCII characters Print(f"okay:.2f is slay") → okay yra variable lygus floatui Print("machine {} to {}".format("slay", "okay")) → ides situs zodzius Print("machine {1} to {0}".format("slay", "okay")) → ides situs zodzius tik atv tvarka Print("machine {title} to {aim}".format(title=tool, aim=goal)) tool ir goal yra variables Print("machine {data[tool]} to {data[goal]}".format(data=mood)) mood yra dictionary Print("only {0:f}% of the {1}".format(0.511, "cows")) Print(f"called {family['dad']}") Print(f'{number1} multi {number2} is {number1 * number2}'), galima ir funkcijas idet tipo: {my_function(10,20)} My_string=Template('data is \$identified') ir my_string.substitute(identifier="lol"),	arba jei lol jau yra okay="lol" tai indetifier=okay) My_string.safe_substitute Classes: Class Rectangle: Def _init_(self, length, width=1): Self.length = length Self.width = width Def size(self): Return self.width * self.length Def info(self): Return f'This rectangle is (self.length) by (self.width)' Print(rectangle_1.size()) → 6 s.isin(list('abcd')) → s.map(lambda x:x in 'abcd') ~s.isin(list('abcd')) → s.map(lambda x:x not in 'abcd') Comprehensions: l2 = [x * 2 if x%2 == 1 else x // 2 for x in l1 if x%3 != 0] d1 = {k: v*3 for k,v in zip(l1,l2) if not (k%2 == 0 or v%2 == 0)} s1 = [x * 2 for x in l1 if not (x%2 == 0)] l1 = [x * 2 if x.lower() in 'aeiou' else x * 3 for x in s1 if x in string.ascii_letters] ir s2 = "".join(l1) result = {x:{y: func(x,y) for y in range(x, 4)} for x in range(1, 4)} → {1: {1: 2, 2: 3, 3: 4}, 2: {2: 4, 3: 5}, 3: {3: 6}}, print(result[1][3])→4 result = {(x, y): func(x,y) for x in range(1, 4) for y in range(x, 4) } print(result) → {(1, 1): 2, (1, 2): 3, (1, 3): 4, (2, 2): 4, (2, 3): 5, (3, 3): 6} print(result[2,3]) → 5 pairs_2=[(num1, num2) for num1 in range(0,2) for num2 in range(6,8)] → [(0,6), (0,7), (1,6), (1,7)] With lambda: L2=list(map(lambda x:x*2 if x%2==1 else x//2, filter(lambda x:x%3!=0, l1))) Generator: Def genfunction(start, stop, step): While (num:=start) <=stop: Yield num Start+=step Return 'end of numbers' mygen=(x for x in range(0,5)) → reikia sklaistuti! To turn into a list the answer: list(genfunction(1,5,2)) Iterating: myiterador = iter([1,3,5]) print(next(myiterador)) 1, kitos eil 3, dar 5 Word="yes", it=iter(word), next (it) → po viena raide ismes 'y' Print(*it) → y e s ----- A=sorted('01234', key=lambda x: str(int(x) % 2 ==1 + x), print("".join(a)) →02413 print(s1.str.upper().str.count('A').mean()) df.isna() → to indicate missing values, df.fillna(value) and df.dropna() → drops rows/columns with missing values	Datetime print(datetime.now().timestamp()) → belekiek sekundziu ismestu d = datetime.now() → 2024-05-21 07:17:08.634938 d = datetime (2024, 1, 10) → 2024-01-10 00:00:00 d = datetime (hour=17, month=5, day=22, second=30, minute = 10, year=2023) d = datetime.now() {d.isoformat()} → 2024-05-22T16:02:10.806118 Day of the week (1=Monday): {d.isoweekday()} → 3 Week of the year: {d.isocalendar()[1]} → 21 #butinai vienetas turi but Year: {d.isocalendar()[0]} → 2024 today = datetime.now() future_day = datetime.now +timedelta(weeks=14, days=2) future_day = datetime.now() + timedelta(minutes=100*24*60) print (future_day.strftime("%Y-%m-%d")) birthday = datetime(1923, 10, 26) age = datetime.now() - birthday print(age) → 36733 days, 7:30:55.168969 age.days =36733, age.seconds =27055 Datetime.strptime('2024\$05%20', "%Y\$%m\$%d") datetime.strptime("2024*141", "%Y*%j") d = datetime.strptime("24\$05%20", "%y\$%m\$%d") abu isprintins: 2024-05-20 00:00:00 d = d.replace(year=2026), d = d.replace(year=d.year - 2) string representation of datetime objects: %Y full year %y year w/o century %B month name %b month nr %d day %A weekday name %w weekday nr %U week nr of the year %j day nr of the year %H hour %M min %S sec If you want to add days or seconds to a datetime object use timedelta jei abu datetime objektai nereikia timedelta naudot, uztenka tiesiog datetime.days ir jau automatskai timedelta veiks datetime'ui reikia kad butu 2023,1,10 o ne 2023/1/10 todel mes naudojame strptime kuris pakeis ji strptime: You provide it with a date/time string and a corresponding format string that matches the structure of the input string to change it into a datetime object: datetime.strptime(date_string, format) strftime is used to format a datetime object into a string according to a specified format: datetime.strftime(format) Dataframe: Df.index → row names Df.columns → column names	If many rows and one column: series with a column name, and vice versa Also can df3.loc[df3.10<155] pasirinks visas eiles kur values in 13 maziau uz 155 Print(df1) - shows first 5 and last 5 rows Print(df1.head(3)) - shows first 3 rows Print(df1.tail(7)) - shows last 7 rows Sorting: Df1=df1.sort_index (axis=0, ascending=False) → isrusiuoja pagal eiliu pavadinimus nuo max iki min axis=1 pagal stulpeliu pavadinimus By values in a row: Df1.sort_values (by='First', ascending, axis=1) By values in a column: Df1.sort_values(by='First', axis=0) pandas: df.index → Index(['first', 'second'], dtype='object') creating pandas: 1) d1 = {'Name': ['A', 'B', 'C'], 'Length': [153, 160, 150], 'Weight': [55, 70, 60]} df1 = pd.DataFrame (d1, index=['First', 'Second', 'Third']) 2) l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]] df1 = pd.DataFrame (l1, index=['First', 'Second', 'Third'], columns=['Name', 'Length', 'Weight']) 3) l1 = [['A', 153, 55], ['B', 160, 70], ['C', 150, 60]] df1 = pd.DataFrame (l1) df1.index = df1.index.map(lambda x: x.upper()) → pervadina rows i upper d1 = {'First': 153, 'Second': 160, 'Third': 150} s1 = pd.Series (d1, name = 'Length') → po sito series apacioj atprintins dar: name=Length l1 = [153, 160, 150] s1 = pd.Series (l1, index=['First', 'Second', 'Third'], name = 'Length') s1 = pd.Series([153, 160, 150], name='Length') s1 = s1.drop(1) → isdropins antra eilute bet jei vel parasyti tai nebedropins (ne kaip su lists) df2=df1.sort_index(axis=0, ascending=False) arba dar galima by='First' df1=df1.T Selection: You can leave out column selection df.loc[row_label, column_label] df.iloc[row_index, column_index] Df.loc[:,:] or df1.iloc[:,] bei df.loc[:,:] or df1.iloc[:,] pasirenka visiskai viska df1.loc[:, 10:12] df1.iloc[:, :1] pasirenka stulpelius nuo 10 iki 12 df1.loc[[3, 0]] df1.iloc[[0,3]] pasirenka eilutes su pavadinimu 3 ir 0 df1.loc[[2]] df1.iloc[[1]] pasirenka antra eilute df1.loc[2] df1.iloc[1] pasirenka antra eil, bet be pavadinimo ir stulpeliu vardai dabar tampa eiluciu vardais ir apacioj Name: 2	df1.loc[[2],[13]] df1.iloc[[1],[1]] pasirenka 2e ir 12st su abiem pavadinimais df1.loc[[2],13] df1.iloc[[-3],[-3]] gaunasi eilute 2 160 ir apacioj Name:13 df1.loc[2,13] df1.iloc[[-3,1]] ismeta tiesiog value (160) s1 = df1.loc[2] print(s1.name) → 2, o su s1 = df1.loc[:,13] → 13 changing a section: df1.loc[[3,0]] = [['E', 100, 40, 20], ['F', 101, 41, 21]] df1[[10,13]][[10]] → selects col 10 and 13 and then 10 print(df1.loc[[True, True, False, False], [False, True, False, True]]) print(df1.loc[df1.loc[:,13] < 155]) → filters which values in column 13 are above 155 and then drops rows which dont fit this description l1 = [11, 2, 3, 14, 5, 16, 7, 18, 29] s1 = pd.Series (l1) s1.index = range(3,30,3) → pakeicia eiluciu pavadinimus print(s1.loc[15]) → suranda eilute su pavadinimu 15 ir paraso jos value Selecting shortcuts: 1) Df.Name → df.loc[:, 'Name'] → Df['Name'] → isprintina eiliu pavadinimus ir values is column su pavadinimu 'Name' ir apacioj Name:Name 2) Df. [['Name']-> df.loc[:, :['Name']] → df1[['Name']] o sitas taip pat tik virsuj su eilutes pavadinimu ir be Name: apacioj 3) df1.Name['First']) kaip ir df1.loc['First','Name'] atprintins tiesiog value 4) df1.Name[['First', 'Second']] kaip ir df1.loc[['First', 'Second'],:,'Name'] atprintins First A ir Second B kitos eilutej ir apacioj Name:name S1[1]-> s1.iloc[1] S1[:2]-> s1.iloc[:2] Broadcasting: Df+=1 or df.A+=1 adds one to all elements or elements in column A df2 += pd.DataFrame({'A':[2, 5, 6], 'B': [3, 4, 7]}) → prie esamu skaiciu pridedi A stulpelyje atitinkamai 2, 5, 6 ir tt df1.A.mean() arba df1.mean()['A'] → returns a series with answers as values and columns name as indexes df2.A = df2.A.map(lambda x: 3 * x) df2.loc[1] = df2.loc[1].map(lambda x: 3 * x) df2 = df2.applymap(lambda x: 3 * x) df1=df1.drop(1) → will drop the row that is called 1 df1=df1.drop(1,axis=1) → will drop the column that is called 1 Creating a new row: Df1.loc['Av'] = df1.apply(lambda x: x.mean(), axis=0) Creating a new column: Df1.loc[:, 'Av'] = df1.apply(lambda x: x.mean(), axis=1) Add a new row: Df1.loc['s']=[1,2,3] Add a new column: Df1.loc[:, 's']=[1,2,3]
---	--	---	---	--

To apply methods intended for strings:
s.str[::-1] -> s.map(lambda x: x[::-1])
s.str.lower(), s.str.count('a')

```
s1 = pd.Series([1:'Aap', 2:'Nota',  
4:'MIES'])  
print(s1.map(lambda x: x[::-1])) ir  
print(s1.str[::-1])--> paA, atoN; galima  
ir x.isupper() ir x.count('a') daryt
```

```
s1 = pd.Series(list('Python')) -> vienoj  
eilutej 0 P, kitoj 1 y ir tt  
print(s1.map(lambda x: x.lower() in  
'aeiou')) -> True/False suraso  
print(s1.str.lower().isin(list('aeiou'))) ->  
same  
print(s1[s1.str.lower().isin(list('aeiou'))  
]) -> paraso 4 o (tik kur true buvo); isin  
only works on lists!
```

```
df1.loc['Av'] = df1.apply(lambda x:  
x.mean(), axis = 0) --> prie df prideda  
eilute av ir paraso vidurkiaus A ir B  
stulpeliu  
df1.loc[:, 'Av'] = df1.apply(lambda x:  
x.mean(), axis = 1) -> tas pat tik su  
eiluciu vidurkais
```

```
adding data: df2 = df1.loc[:,] ir df2.loc[:,  
'C'] = 99 (visose eilutese parasys 99,  
arba galima 99,99,99)  
pakeisti: df2 = df1.loc[:,] df2.loc[0] = 99  
(jei i stulpelius nori idet kad nulinej  
eilutej abiejuose stulpeliuose  
pasikeistu i 99 values)
```

```
print(pd.concat([df1, df2])) --> prides  
zemyn i eilutes  
print(pd.concat([df1, df2], axis = 1)) -->  
prides sonan
```

```
print(df.loc[df['Height'] > 170, ['Player',  
'Age']]): prints the 'Player' and 'Age'  
columns for players whose 'Height' >  
170.  
df[['Player', 'Age']] selects the columns  
'Player' and 'Age' from the DataFrame  
df  
jei viena column: df['column_name']  
print(df[1, 'Age']) --> error nes row  
names negalima det su sita funkcija
```

```
S1=S1.drop(1) -> will drop the row that  
is called 1  
S1.name -> get the name of the series
```

Datacamp:

```
Credit_records.price.sum()  
Plt.plot(ransom['letter'],  
ransom['frequency'])  
Selecting the values of a column:  
1) Suspect=credit_records['suspect']  
2) Price=credit_records.price  
Credit_records.price > 20  
credit_records[credit_records.price >  
20]
```

```
Egzo uzd:  
data['local'] =  
pd.Series([email.split("@")[0] for email  
in args])  
return pd.DataFrame(data)
```